HARNESS · CONTINUOUS DELIVERY · UNIFIED PIPELINE MODEL

CDS-122512 Test Evidence Report

Manual verification of SteadyStateCheck pre/post service-hook execution for the Unified pipeline model, covering Helm and Kubernetes deploy steps across 13 scripted cases.

CDS-122512

SteadyStateCheck pre/post service hooks — Unified Helm & K8s

Scope: hook injection, ordering, stop-on-failure, timing, rollback isolation, mixed hook-phase independence

Affected components

harness-core · helm-plugin · kubernetes-plugin · template-library

Flag: FeatureName.CDS_UNIFIED_SERVICE_HOOKS

13	13	2	4
CASES RUN	PASSED	RUNTIMES (HELM, K8S)	REPOS CHANGED

Methodology. Each case is a standalone Harness Unified-model pipeline executed against a live delegate and Kubernetes cluster. Pre/post hook execution is proven with filesystem stamp files (timestamps or run counters) written by the hook scripts into the step workspace, inspected by a trailing `assert` step or by the deploy step's own log output where no assert step is needed. Cases 1–7 and 13 exercise the Kubernetes rolling-deploy path; cases 8–12 exercise the Helm basic-deploy path with the identical hook contract.

Author: Raj Rakshit Shukla

Manual test campaign · 2026-09-28 – 2026-09-30

account: HingerDemo

01 Feature Under Test

env-var contract shared by Helm & K8s

```
# unified service config -> sweeping output -> sorted by order -> serialized PLUGIN_PRE_STEADY_STATE_HOOK_SCRIPTS="
<script1>,<script2>,..." PLUGIN_POST_STEADY_STATE_HOOK_SCRIPTS="<script1>,..." # both plugins run each script sequentially
via `sh -c`, stop at first non-zero exit # gated behind FeatureName.CDS_UNIFIED_SERVICE_HOOKS
```

REPO	ROLE
harness-core	Reads SteadyStateCheck hooks off the unified service sweeping output, filters/sorts by `order`, injects the two env vars for both service types.
helm-plugin	helm-deploy plugin runs pre-hook scripts immediately before, and post-hook scripts immediately after, the steady-state poll.
kubernetes-plugin	k8s-steady-check plugin runs the identical pre/post contract around the rollout steady-state poll.
template-library	`helmDeployAction` template exposes `pre_steady_check_hook_scripts` / `post_steady_check_hook_scripts` array inputs, mirroring the existing `k8sRollingDeployStep` pattern.

02 Run Inventory

13 cases, chronological

#	RUNTIME	VARIATION UNDER TEST	VERDICT
1	K8s	Happy path — pre/post stamps present, pre ≤ post	PASS
2	K8s	Pre-hook fails — deploy fails, post must never run	PASS
3	K8s	Post-hook fails — rollout succeeds, step still reported FAILED	PASS
4	K8s	Multi pre-hook chain, hook 2 fails — stop-on-first-failure	PASS
5	K8s	No hooks configured — regression control	PASS
6	K8s	Long-running steady-state poll (~60s) — post waits for real poll, not fired early	PASS
7	K8s	Explicit rollback after deploy — rollback must not re-trigger hooks	PASS
8	Helm	Happy path — Helm mirror of case 1	PASS
9	Helm	Pre-hook fails — Helm mirror of case 2	PASS
10	Helm	Post-hook fails — Helm mirror of case 3	PASS
11	Helm	Multi pre-hook chain, hook 2 fails — Helm mirror of case 4	PASS
12	Helm	No hooks configured — Helm regression control	PASS
13	K8s	Mixed hook phases (FetchFiles + TemplateManifest + SteadyStateCheck) on one deploy — phase-routing regression guard	PASS

03 Per-Case Results

Kubernetes — cases 1–7

Case 1

PASS

K8s · happy path

What: Rolling deploy with both PRE and POST SteadyStateCheck hooks configured.

Why: Confirm each hook fires exactly once, pre before post, when nothing fails.

Expected: pre.stamp & post.stamp both written, pre ≤ post.

Actual:

```
PASS: pre (1790780942) <= post (1790780943)
```

Case 2

PASS

K8s · pre-hook fails

What: PRE hook configured to exit 1 immediately.

Why: A failing pre-hook must block the deploy and the post-hook must never run.

Expected: deploy step FAILED; no post-after-prefail stamp.

Actual:

```
deploy step FAILED on pre-hook; no post stamp written;
downstream assert skipped (default no-override failure-
strategy).
```

Case 3

PASS

K8s · post-hook fails

What: Rollout succeeds normally; POST hook configured to exit 1.

Why: A failing post-hook must still fail the step even on a healthy rollout — hooks are part of the step's success contract.

Expected: rollout completes; step still reported FAILED.

Actual:

```
"PRE-STEADY-HOOK RAN OK" → rollout healthy → post hook exit
1 → step FAILED
```

Case 4

PASS

K8s · multi-hook order

What: Three PRE hooks in declared order; hook2 exits 1.

Why: Confirm hooks execute in order and the chain stops at the first failure.

Expected: hook1 + hook2 stamps exist, hook3 stamp absent, deploy FAILED.

Actual:

```
hook1 ran → hook2 ran, FATAL hook 2/3 exit 1 → hook3 never
ran
```

Case 5

PASS

K8s · no hooks (control)

What: Same deploy, service has no SteadyStateCheck hooks configured.

Why: Regression control — feature must be a no-op when unused.

Expected: identical to pre-fix behavior, plain PASS.

Actual:

```
no PRE/POST_STEADY_STATE_HOOK_SCRIPTS params present;
rollout + steady-state check completed normally
```

Case 6

PASS

K8s · long-running poll timing

What: Readiness probe delayed ~60s; PRE & POST hooks each stamp a timestamp.

Why: Prove POST fires only after the real steady-state poll resolves, not immediately.

Expected: post − pre delta ≥ 45s.

Actual:

```
pre=1790781846 post=1790781898 — delta=52s
```

Case 7

PASS

K8s · explicit rollback

What: Deploy (hooks fire once), followed by an explicit Rolling Rollback step.

Why: Rollback must not route through the SteadyStateCheck hook branch — hooks must not re-fire.

Expected: pre/post counters stay at exactly 1 line each after rollback.

Actual:

```
pre_lines=1 post_lines=1 — PASS: rollback did not re-trigger
hooks
```

Helm & mixed-phase cases

cases 8–13

Case 8

PASS

Helm · happy path

What: Helm mirror of case 1 — helmBasicDeployStep with PRE+POST hooks.

Why: Confirm the identical hook contract works on the Helm deploy path.

Expected: pre.stamp & post.stamp both written, pre ≤ post.

Actual:

```
hooks fired correctly from the first run; re-run after test-fixture fix (see §04) confirmed pre ≤ post
```

Case 9

PASS

Helm · pre-hook fails

What: Helm mirror of case 2 — PRE hook exits 1.

Why: Confirm Helm stops the deploy on a failing pre-hook, mirroring K8s exactly.

Expected: deploy step FAILED; no post-after-prefail stamp.

Actual:

```
deploy log: pre-steady-state-check hook failed; assert skipped (default failure-strategy, same as case 2)
```

Case 10

PASS

Helm · post-hook fails

What: Helm mirror of case 3 — rollout succeeds, POST hook exits 1.

Why: Confirm Helm reports FAILED on a post-hook failure despite a healthy release.

Expected: release rolls out; step reported FAILED.

Actual:

```
[FATAL] post-steady-state-check hook 1/1 failed: exit status 1
```

Case 11

PASS

Helm · multi-hook order

What: Helm mirror of case 4 — three PRE hooks, hook2 fails.

Why: Confirm ordering + stop-on-first-failure matches K8s on the Helm plugin.

Expected: hook1+hook2 stamps exist, hook3 absent.

Actual:

```
[FATAL] pre-steady-state-check hook 2/3 failed: exit status 1; hook3 never ran
```

Case 12

PASS

Helm · no hooks (control)

What: Helm mirror of case 5 — no hooks configured.

Why: Regression control for the Helm path.

Expected: unchanged pre-fix behavior, plain PASS.

Actual:

```
[INFO] Helm basic Deploy successfully finished
```

Case 13

PASS

K8s · mixed hook phases

What: One deploy exercising FetchFiles, TemplateManifest, and SteadyStateCheck hooks together.

Why: Regression guard for ServiceHooksStep.fetchHooksSweepingOutputForPhase — adding steady-state node IDs without an explicit branch could misroute POST_STEADY_STATE hooks through the FetchFiles sweeping output.

Expected: all four stamps present, each phase independent.

Actual:

```
PASS: mixed-fetch, mixed-template, mixed-steady-pre, mixed-steady-post — all stamps present
```

04 Coverage Matrix

behaviour → cases

BEHAVIOUR	CASES	STATUS
Hook injection & execution (happy path)	1, 8	COVERED
Pre-hook failure blocks deploy, post never runs	2, 9	COVERED
Post-hook failure still fails the step after a successful rollout	3, 10	COVERED
Multi-hook ordering + stop-on-first-failure	4, 11	COVERED
No-hooks regression (behavior unchanged when unconfigured)	5, 12	COVERED
Post-hook timing follows the real steady-state poll, not fired early	6	COVERED
Rollback does not re-trigger SteadyStateCheck hooks	7	COVERED
SteadyStateCheck hooks independent of FetchFiles/TemplateManifest hook phases	13	COVERED

Not a product defect — test-fixture bug (Case 8). The first Helm happy-path run stalled: the readiness probe got `connection refused` on port 8080. Root cause was the test chart's `values.yaml` setting `containerPort: 8080` while its image (`nginx:1.25`) only listens on port 80 — the parallel K8s fixture already used port 80 for the same image. Hook injection itself worked correctly even during the stalled run (both env vars carried real content, the pre-hook printed and succeeded). Fixed `values.yaml` to `containerPort: 80`, pushed, re-ran: PASS. No code change required.

Reconstructed from live Harness Unified-pipeline executions against a real delegate/cluster during the manual test campaign. Evidence snippets are condensed to the lines that establish pass/fail; full execution logs are available in the originating pipeline runs.